Notas del orador — protoST — FAST 2026

Exportadas de `deck.html` (el mismo contenido que el deck publicado). Los PDF de `slides/` no llevan notas.

Diapositiva 1 — protoST

Presentarse en una frase. Decir desde el principio que esto es un experimento y que la charla busca gente curiosa, no usuarios.

Diapositiva 2 — Un experimento que busca entusiastas.

Fijar el tono: nada de marketing. Si algo no funciona, lo digo. Todo lo que aparece tiene un test o un informe detrás.

Diapositiva 3 — ¿Les suena?

Plantearlas como preguntas y dejar que el público responda. No afirmar sus dolores por ellos. No prometer que protoST los resuelve: abre otra forma de intentarlo.

Diapositiva 4 — protoCore: un núcleo, varios lenguajes

Tres propiedades, un ejemplo de cada una si preguntan. protoCore es un proyecto aparte; protoST es su demostrador.

Diapositiva 5 — Un objeto mutable es una referencia a una instantánea inmutable

Modificar instala una instantánea nueva; quien tenía la anterior la sigue viendo entera. Por eso pasar objetos entre hilos no necesita copias ni locks.

Diapositiva 6 — Demo 1 · el código que ya conocen

Correr ./run_demo.sh 01-familiar-code.st. Mostrar el archivo antes. Si falla: scriptreplay de recordings/01. Remarcar que new envía initialize; la última línea imprime el total (270): un script muestra solo lo que imprime.

Diapositiva 7 — Cualquier objeto puede ser un actor

wait, thenDo:, whenAll:. Un error dentro del actor vuelve con su clase al que espera. Un actor que espera no atiende otros mensajes; si dos se esperan mutuamente, protoST lo reporta como error.

Diapositiva 8 — El mismo cálculo, en un hilo y en doce actores

Correr ./run_demo.sh 02-actors-parallelism.st. La última línea (time speedup) es la aceleración: decir la que aparezca en pantalla, no una de memoria. Depende de la máquina y de su carga: con protoCore 2.14.6, en la notebook de la charla, mediana 3,8× (3,3–4,2×), un hilo en unos 925 ms y doce actores en unos 245 ms. Es menos que las 4,4× de antes porque 2.14.6 hizo un 45 % más rápido el código de un hilo (y un 35 % el de doce actores): ver preguntas-dificiles.md. El código del cálculo no cambió: cambió quién lo ejecuta.

Diapositiva 9 — Mundos que podría abrir

Ser explícito con la tercera: funciona embebiendo los dos runtimes en un programa, y lo probado es una dirección: protoScala lee y escribe objetos de protoST. La inversa (protoST usando objetos o módulos de otro runtime) todavía no funciona, y los binarios instalados no se cargan entre sí. docs/INTEROP.md §0 dice exactamente qué funciona. Seguir con la demo 4.

Diapositiva 10 — Otro programa entrega los datos; protoST los procesa y los publica

Correr ./run_demo.sh 04-connected-twin.st. Mostrar el código: los nombres son los de Pharo (Stdio, asFileReference, writeStreamDo:). Otro lenguaje entrega los datos, protoST los procesa con actores y los expone por HTTP; nada de eso necesitó una imagen. Si falla: scriptreplay de recordings/04.

Diapositiva 11 — Cada benchmark verifica su resultado

Las cifras salen de los informes; si no están en un informe, no se dicen. El harness falla si un benchmark no reporta el resultado correcto: así una caída nunca parece una mejora. Todas las cifras de la diapositiva: informe del 12/10 sobre protoCore 2.14.6 (el binario publicado de protoST sobre 2.14.6 y sobre 2.14.5, medianas de 14 corridas intercaladas con pares A/A; CPython 3.14 en la misma máquina). El rango 14–20× es honesto: con N chicos el trabajo es la resta de dos tiempos parecidos y el ruido la mueve; el proceso completo da 3,2–3,3× de media geométrica. protoCore 2.14.6 frente a 2.14.5: la demo 2 un 45 % más rápida en un hilo y un 35 % con doce actores, las excepciones un 7 %, el resto igual dentro del ruido; con el techo fijo de 640 MB, el pico de K5 baja de 1165 a 690 MB. 0.6.2 frente a 0.6.1 (informe del 12/10 sobre el heap adaptativo): actores 16–43 % más rápidos, entre un quinto y la mitad de la memoria, pausas de 0,27 ms como máximo; pero el pico de memoria ya no está acotado (un bucle que asigna llegó a 0,88–3,5 GB).

Diapositiva 12 — Todo escrito y verificado

Leer los titulares. El mensaje: no escondemos nada; si algo sorprende y no está en el capítulo 14, es un bug y lo queremos saber.

Diapositiva 13 — Revisiones adversariales, antes de esta charla

Contar el cuelgue intermitente como ejemplo de método: en vez de reintentar, capturar la pila del proceso colgado. Los ≈1100 programas son de la auditoría adversarial (docs/STATUS.md); los tests son la suite de ctest, otra cosa. La revisión de la E/S (fuente: protoIO/docs/specs/2026-09-30-protoio-design.md; correcciones en el commit c6beb6e de protoST) encontró, entre otros, que un programa hijo que no leía su entrada mataba al intérprete (SIGPIPE), que varios actores leyendo la entrada estándar mezclaban líneas y un mutex del pool tomado durante una asignación. También sacó a la luz un defecto de protoCore al crear hilos, corregido en protoCore 2.6.1. 1101 es la suite de 0.6.5 con protoCore 2.14.6 en Linux (0.5.0 tenía 1068); la CI corre 1100 en Linux y macOS y 1097 en Windows. En 0.6.3 una advertencia de GCC 15 destapó un defecto latente: el despacho con goto computado no liberaba los pins transitorios del recolector; corregido, con su test.

Diapositiva 14 — Escriban un programa como lo escribirían ustedes.

Cerrar con la invitación concreta y abrir preguntas. Tener a mano preguntas-dificiles.md. La instalación son dos .deb (protoCore 2.14.6 y protoST 0.6.5) adjuntos a los releases de GitHub; en Windows, dentro de WSL2 con Ubuntu 24.04. Las cuatro demos vienen en el paquete, en /usr/share/protoST/talks/2026-10-15-fas/demos. El .deb de protoST 0.6.5 exige protoCore 2.14.6; compilado desde las fuentes necesita 2.11 o posterior, y compila también en forma nativa en Windows; todavía no hay paquete publicado para Windows.

Diapositiva 15 — Demo 3 · gemelo digital (módulo) · salida abreviada

Correr ./run_demo.sh 03-digital-twin.st. El tiempo por ciclo ronda 50 ms, no 150, porque los tres sensores responden en paralelo. Recorrer el código si hay tiempo.

Diapositiva 16 — Celdas, instantáneas y un GC que se adelanta

Módulo opcional de 10 minutos. Dibujar en pizarra la instantánea atómica si hay preguntas. El recolector es el de protoCore 2.14.6, cuya lista de trabajo del marcado tiene un tamaño fijo (8 MiB): con el techo fijo de 640 MB, el pico de un arreglo de 500 000 elementos bajó de 1165 a 690 MB (informe del 12/10 sobre 2.14.6). Desde 0.6.2 protoST usa el controlador adaptativo del heap de protoCore (hasta 0.6.1, un techo fijo de 640 MB): programas de actores 16–43 % más rápidos con entre un quinto y la mitad de la memoria; el precio es que el pico ya no está acotado (0,88–3,5 GB en un bucle que asigna; PROTOCORE_HEAP_LIMIT_CELLS lo fija). Hubo que corregir algo propio: cada pausa stop-the-world recorría entero un arreglo de frames de 1 M de entradas por hilo (pausas de hasta 20–31 ms con seis a doce workers); ahora el arreglo crece bajo demanda y las pausas no pasan de 0,27 ms en los benchmarks (KNOWN_ISSUES.md, K5 y K6).

Diapositiva 17 — ¿Qué les sería útil primero?

Preguntar al público y anotar. La hoja de ruta está en docs/ROADMAP.md.